TP1: Basic Python Programming

Programming for Data Science · Chapter 1 · Basic Python & Libraries

Problems at a Glance

#	Problem	Level	Key topics
1	Student Profile Card	Easy	variables, types, f-strings
2	Time Converter	Easy	// and % operators
3	Temperature Converter	Easy	functions, default arguments
4	Type-Conversion Detective	Medium	implicit & explicit casting
5	Letter Grade	Medium	if / elif / else
6	Leap Years and Days in a Month	Medium	logical operators, functions
7	Digit Tools	Medium	while loops, // and %
8	Flexible Functions: *args, **kwargs and lambda	Medium	*args, **kwargs, lambda
9	Electricity Bill with Progressive Tiers	Hard	conditions, min/max, functions
10	Prime Numbers and Goldbach's Conjecture	Hard	nested loops, break, functions

10 problems: 3 easy, 5 medium, 2 hard.

Objectives

By the end of this TP you should be able to:

- Store information in variables and check their types with `type()`.
- Use arithmetic, comparison, logical and membership operators correctly.
- Convert between `int`, `float`, `str` and `bool`, and handle bad input.
- Control the flow of a program with `if / elif / else`, `for` and `while`.
- Package logic into reusable functions with default, `*args` and `**kwargs` parameters.

Instructions

- Work in VS Code with the course virtual environment (`Codes\.venv`). Every library you need is in `requirements.txt`.
- Write all your answers in **one script** named `tp1_<student-id>.py`, with one commented section per problem (`# Problem 1`, `# Problem 2`, …).
- Each problem is marked **Easy**, **Medium** or **Hard**. Solve them in order: the later problems reuse ideas from the earlier ones.
- An **expected output** box shows what a correct program prints. A line `...` means part of the output was left out.
- When a question asks *why* or *explain*, answer in a short comment (`# ...`) in your script.
- Try each problem on your own before asking for help. Reading the documentation (`help(...)`) is part of the exercise.

Problems

Problem 1. Student Profile Card *(Easy)*

Topics: variables, types, f-strings

Create five variables that describe a student: `name` (str), `age` (int), `major` (str), `gpa` (float) and `is_scholar` (bool).

(a) Print a profile card exactly like the one below. The GPA must always show **2 decimals** (hint: `f"{gpa:.2f}"`).

(b) Below the card, print every value followed by the name of its type, using `type(value).__name__`.

(c) Re-assign `age = "20"` and print `type(age)` again. What does this tell you about *dynamic typing*?

```
EXPECTED OUTPUT
==============================
Name    : Sok Dara
Age     : 20
Major   : Data Science
GPA     : 3.67
Scholar : True
==============================
'Sok Dara' -> str
20 -> int
'Data Science' -> str
3.67 -> float
True -> bool
```

Problem 2. Time Converter *(Easy)*

Topics: // and % operators

A video lasts `total_seconds = 9875` seconds. Convert it into hours, minutes and seconds using **only** the operators `//` (floor division) and `%` (modulo).

(a) Compute `hours`, `minutes` and `seconds`, then print the sentence shown below.

(b) Check your answer: rebuild the total from the three parts and print whether it equals `total_seconds`.

(c) Test your code with 3600, 59 and 86399 seconds.

```
EXPECTED OUTPUT
9875 seconds = 2 h 44 min 35 s
Check: True
```

> ***Hint:*** *One hour has 3600 seconds. After removing the whole hours with %, what is left is less than one hour.*

Problem 3. Temperature Converter *(Easy)*

Topics: functions, default arguments

Write two functions: `celsius_to_fahrenheit(c, round_to=1)` and `fahrenheit_to_celsius(f, round_to=1)`. Use the formula **F = C × 9/5 + 32** and return the result rounded with `round(value, round_to)`.

(a) Use a `for` loop with `range(0, 101, 20)` to print the conversion table below.

(b) Convert 98.6 °F (normal body temperature) to Celsius.

(c) Convert 37 °C to Fahrenheit and back again. Do you get 37.0?

(d) Call one function with `round_to=3`. Why is a default argument useful here?

```
EXPECTED OUTPUT
  °C  |   °F
------+-------
    0 |   32.0
   20 |   68.0
   40 |  104.0
   60 |  140.0
   80 |  176.0
  100 |  212.0
98.6 °F = 37.0 °C
```

Problem 4. Type-Conversion Detective *(Medium)*

Topics: implicit & explicit casting

Part A. Copy the table into your notebook. **Predict** the result of each expression first, then check it in Python.

Expression	Your prediction	Python's answer
`int("7") + 3.5`		
`str(10) * 3`		
`bool(""), bool(" ")`		
`int(3.99)`		
`float("1e3")`		
`int(True) + float("2.5")`		
`round(2.5), round(3.5)`		
`type(10 / 2)`		

Part B. Write a function `safe_int(text)` that turns a string into an `int`. It must accept spaces around the number, negative numbers and decimals (`"3.9"` → 3), and return `None` when the text is not a number.

```
PYTHON
for sample in ["42", " -8 ", "3.9", "abc", ""]:
    print(repr(sample), "->", safe_int(sample))
```

```
EXPECTED OUTPUT
'42' -> 42
' -8 ' -> -8
'3.9' -> 3
'abc' -> None
'' -> None
```

Part C. Ask the user for two numbers with `input()`, convert them with `safe_int()` and print their sum, or a friendly message if one of them is not a number.

> ***Hint:*** *`str.strip()`, `str.isdigit()` and a `try / except ValueError` block are all you need.*

Problem 5. Letter Grade *(Medium)*

Topics: if / elif / else

Write a function `letter_grade(score)` that returns a letter according to this scale. Any score below 0 or above 100 must return `"Invalid"`.

Score	≥ 97	90 – 96.9	80 – 89.9	70 – 79.9	60 – 69.9	< 60
Grade	A+	A	B	C	D	F

(a) Test your function with the list `[99, 90, 85.5, 72, 60, 59.9, -5, 101]` and print the results aligned as below.

(b) Why must the test for **A+** come *before* the test for **A**? What happens if you swap them?

```
EXPECTED OUTPUT
   99 -> A+
   90 -> A
 85.5 -> B
   72 -> C
   60 -> D
 59.9 -> F
   -5 -> Invalid
  101 -> Invalid
```

Problem 6. Leap Years and Days in a Month *(Medium)*

Topics: logical operators, functions

A year is a **leap year** if it is divisible by 4, *except* century years (divisible by 100), which are leap years only if they are also divisible by 400.

(a) Write `is_leap(year)` that returns `True` or `False` using **one** boolean expression with `and` / `or`.

(b) Write `days_in_month(month, year)` that returns 28, 29, 30 or 31, or `None` if the month is not between 1 and 12. Use `in` to test the 30-day months (4, 6, 9, 11).

(c) Print whether 1900, 2000, 2024 and 2026 are leap years.

(d) Count the leap years from 2000 to 2100 (both included) with a loop.

```
EXPECTED OUTPUT
1900 not leap
2000 leap
2024 leap
2026 not leap
Feb 2024: 29 | Feb 2026: 28 | Apr: 30 | month 13: None
Leap years from 2000 to 2100: 25
```

Problem 7. Digit Tools *(Medium)*

Topics: while loops, // and %

Without converting numbers to strings, write three functions that work digit by digit with a `while` loop:

- `digit_sum(n)` adds up the digits: 2026 → 2 + 0 + 2 + 6 = 10.
- `reverse_number(n)` reverses the digits: 2026 → 6202.
- `is_palindrome(n)` returns `True` when a number reads the same in both directions (12321).

(a) Test the three functions with 2026, 12321 and 9875.

(b) Count how many palindromes there are from 10 to 999.

```
EXPECTED OUTPUT
2026: digit sum = 10, reversed = 6202, palindrome = False
12321: digit sum = 9, reversed = 12321, palindrome = True
9875: digit sum = 29, reversed = 5789, palindrome = False
Palindromes from 10 to 999: 99
```

Hint: *`n % 10` is the last digit and `n // 10` removes it. Repeat while `n > 0`.*

Problem 8. Flexible Functions: *args, **kwargs and lambda *(Medium)*

*Topics: *args, **kwargs, lambda*

(a) Write `describe(*numbers, decimals=2, label="Data")` that accepts **any number** of values, prints their count, min, max and mean (rounded to `decimals`), and returns them as a tuple. With no values it prints `"<label>: no values"` and returns `None`.

(b) Write `make_tag(text, **style)` that builds an HTML-like string from keyword arguments: `make_tag("Hello", color="red", size=14)` returns `<span color="red" size="14">Hello</span>`.

(c) Write two lambdas: `square` (returns x^2) and `is_adult` (returns `True` when age ≥ 18).

```
PYTHON
describe(12, 7, 30, 4)
describe(3.14159, 2.71828, decimals=3, label="Constants")
describe()
print(make_tag("Hello", color="red", size=14))
print(square(9), is_adult(17), is_adult(21))
```

```
EXPECTED OUTPUT
Data: n=4 min=4 max=30 mean=13.25
Constants: n=2 min=2.71828 max=3.14159 mean=2.93
Data: no values
<span color="red" size="14">Hello</span>
81 False True
```

Problem 9. Electricity Bill with Progressive Tiers *(Hard)*

Topics: conditions, min/max, functions

An electricity company charges each kWh according to the **tier** it falls in, like income tax. A fixed service fee of **2,000 riel** is added, then **10 % VAT** is applied to the total.

Tier	Consumption	Price
1	first 50 kWh (0 – 50)	480 riel / kWh
2	next 150 kWh (51 – 200)	610 riel / kWh
3	everything above 200 kWh	730 riel / kWh

Example: 120 kWh = 50 × 480 + 70 × 610 = 66,700 riel; + 2,000 fee = 68,700; + 10 % VAT = **75,570 riel**.

(a) Write `electricity_bill(kwh)` that returns the bill in riel, rounded to a whole number, or `None` for negative input.

(b) Print a table for 0, 35, 50, 120, 200 and 350 kWh, with thousands separators (`f"{bill:,}"`) and the value in USD (1 USD = 4,100 riel).

(c) Ask the user for their monthly consumption and print their bill.

```
EXPECTED OUTPUT
   0 kWh ->     2,200 riel  (≈   0.54 USD)
  35 kWh ->    20,680 riel  (≈   5.04 USD)
  50 kWh ->    28,600 riel  (≈   6.98 USD)
 120 kWh ->    75,570 riel  (≈  18.43 USD)
 200 kWh ->   129,250 riel  (≈  31.52 USD)
 350 kWh ->   249,700 riel  (≈  60.90 USD)
```

Hint: *The kWh in tier 2 is `min(max(kwh - 50, 0), 150)`. Try to find a similar expression for tiers 1 and 3.*

Problem 10. Prime Numbers and Goldbach's Conjecture *(Hard)*

Topics: nested loops, break, functions

(a) Write `is_prime(n)`. A prime is a whole number ≥ 2 whose only divisors are 1 and itself. For speed, only test divisors `d` while `d * d <= n`.

(b) Print all primes below 50 on one line (use `print(..., end=" ")`), then count the primes below 100.

(c) **Goldbach's conjecture** says every even number ≥ 4 is the sum of two primes. Write `goldbach(n)` that returns the first pair `(p, q)` with `p + q == n`, and print the pairs for 4, 6, …, 30.

(d) Use a loop with `break` to check the conjecture for every even number from 4 to 10,000.

```
EXPECTED OUTPUT
Primes below 50: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
Number of primes below 100: 25
4 = 2 + 2
6 = 3 + 3
8 = 3 + 5
...
30 = 7 + 23
Goldbach holds for every even number from 4 to 10000: True
```

Hint: *Why is it enough to test divisors up to $\sqrt{n}$? If $n = a \times b$ and both a and b were bigger than $\sqrt{n}$, then $a \times b$ would be bigger than n.*